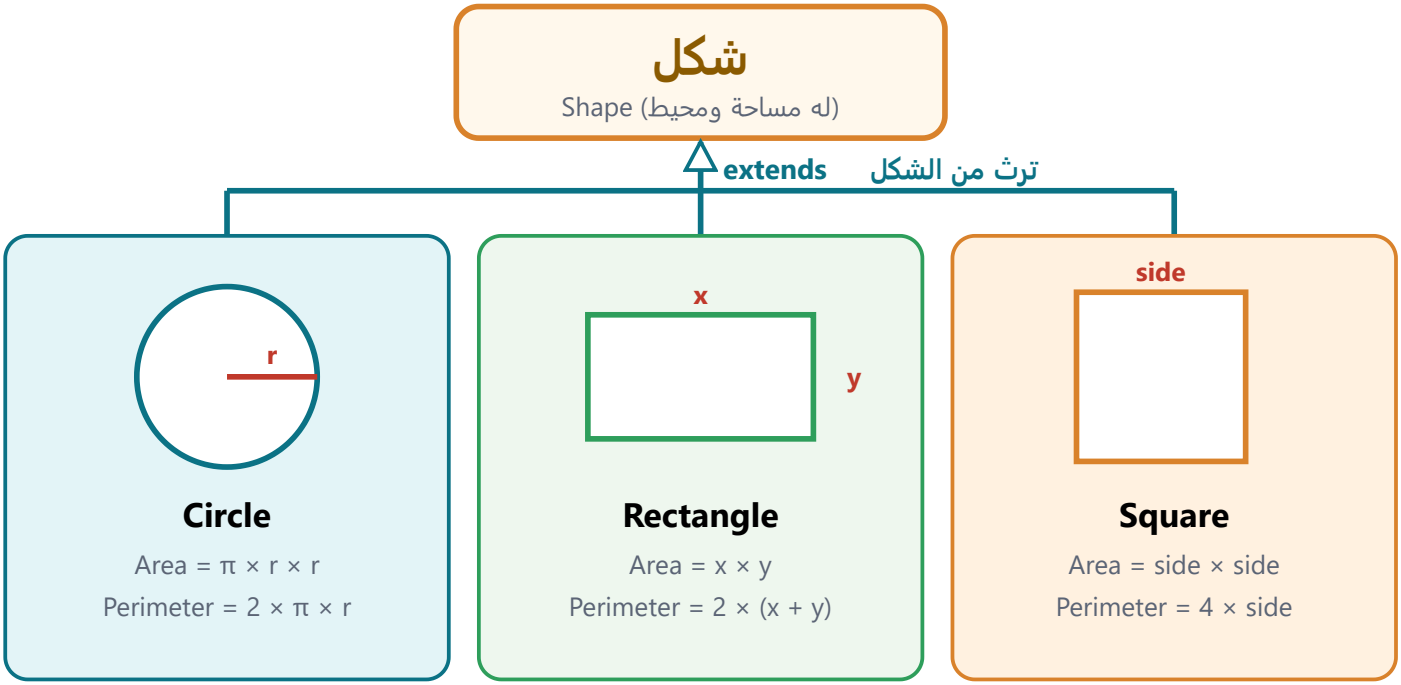
الأشكال الهندسية: من الواقع إلى الكود

1 نبدأ من الواقع: ماذا نعرف عن الأشكال؟

💡 **فكرة:** تخيّلوا أني سأطلب منكم رسم باب غرفتكم، وعجلة سيارة، وبلاطة أرض. ماذا رسمتم؟ مستطيلاً ودائرة ومربعاً. هذه كلها **أشكال هندسية**.

قبل أن نكتب سطراً واحداً من Java، نسأل أنفسنا سؤالين عن الأشكال في الواقع:

1. **ما الذي يُميّز كل شكل عن غيره؟** (ماذا أحتاج لأصف هذا الشكل بالضبط؟)
2. **ما الذي تشترك فيه كل الأشكال؟**



الشكل	ما أحتاجه لأصفه (يميّزه)	المساحة	المحيط
دائرة	نصف القطر `r`	π × r × r	2 × π × r
مستطيل	الطول `x` والعرض `y`	x × y	2 × (x + y)
مربع	طول الضلع `side`	side × side	4 × side

مثال بأرقام	المساحة	المحيط
دائرة نصف قطرها 5	78.54	31.42
مستطيل 4 × 6	24	20
مربع ضلعه 3	9	12

الملاحظة الذهبية: **كل شكل، مهما كان نوعه، له مساحة ومحيط.** لكن **طريقة الحساب تختلف** من شكل لآخر. **المشترك هو "الفكرة" (كل شكل له مساحة)، والمختلف هو "الطريقة" (كيف أحسبها).**

❓ **فكّر:** لو أعطيتكم شكلاً لا تعرفون نوعه، هل أستطيع أن أسألكم "ما مساحته؟" نعم! كل شكل يستطيع الجواب، وكل شكل يجيب بطريقته. **هذه هي الفكرة كلها.**

2 تحليل الواقع: كيف نعرف ما هو الصف وما هي الخاصية وما هو التابع؟

المشكلة: أمامنا شيء من الواقع (دائرة) ونريد أن نكتب عنه برنامجاً. ماذا نكتب بالضبط؟ كيف نعرف أن "الدائرة" صف، وأن "نصف القطر" خاصية، وأن "حساب المساحة" تابع؟

الحل: نصف الواقع بجملة عادية، ثم نصنّف الكلمات المهمة فيها إلى ثلاثة أنواع. هذه الجملة تصف الدائرة:

> **الدائرة** لها **نصف قطر** ، ونستطيع **حساب المساحة** و **حساب المحيط** و **طباعة المعلومات**

لاحظ الألوان: كل لون هو نوع من الكلمات في البرمجة.

اللون	نوع الكلمة	السؤال الذي أسأله لأعرف	تصبح في Java
أزرق	شيء كامل له أنواع كثيرة	هل يوجد منه **كثير**؟ (دوائر كثيرة، كل واحدة بحجم)	**صف** (Class)
أخضر	معلومة عن الشيء	هل هي رقم أو كلمة **أحتفظ بها** عن الشيء؟ (نصف القطر: رقم أحتاجه دائماً)	**خاصية** (Attribute)
أحمر	عمل أطلبه من الشيء	هل هو **فعل**؟ (احسب، اطبع)	**تابع** (Method)

نحوّل الجملة إلى كود

نتيجة التحليل

```
class Circle {                          // الأزرق ← صف
    double radius;                      // الأخضر ← خاصية (معلومة نحتفظ بها)
    double getArea()          { ... }   // الأحمر ← تابع (حساب المساحة)
    double getCircumference() { ... }   // الأحمر ← تابع (حساب المحيط)
    void   printInfo()        { ... }   // الأحمر ← تابع (طباعة المعلومات)
}
```

نجرّب على شكل ثانٍ: المستطيل

> **المستطيل** له **طول** و **عرض** ، ونستطيع **حساب المساحة** و **حساب المحيط**

الكلمة	نوعها	في Java
المستطيل	أزرق (شيء كامل)	`class Rectangle`
طول، عرض	أخضر (معلومات)	`double x;` و `double y;`
حساب المساحة، حساب المحيط	أحمر (أعمال)	`getArea()` و `getCircumference()`

وأين الغرض (Object)؟

كل ما سبق كلام عن **النوع** (الصف). أما عندما نقول **"دائرة نصف قطرها 5"** فنحن نتكلم عن دائرة **محددة** موجودة أمامنا، وهذا هو **الغرض**: `new Circle(5)` . (سنفصّل الفرق في الخطوة القادمة.)

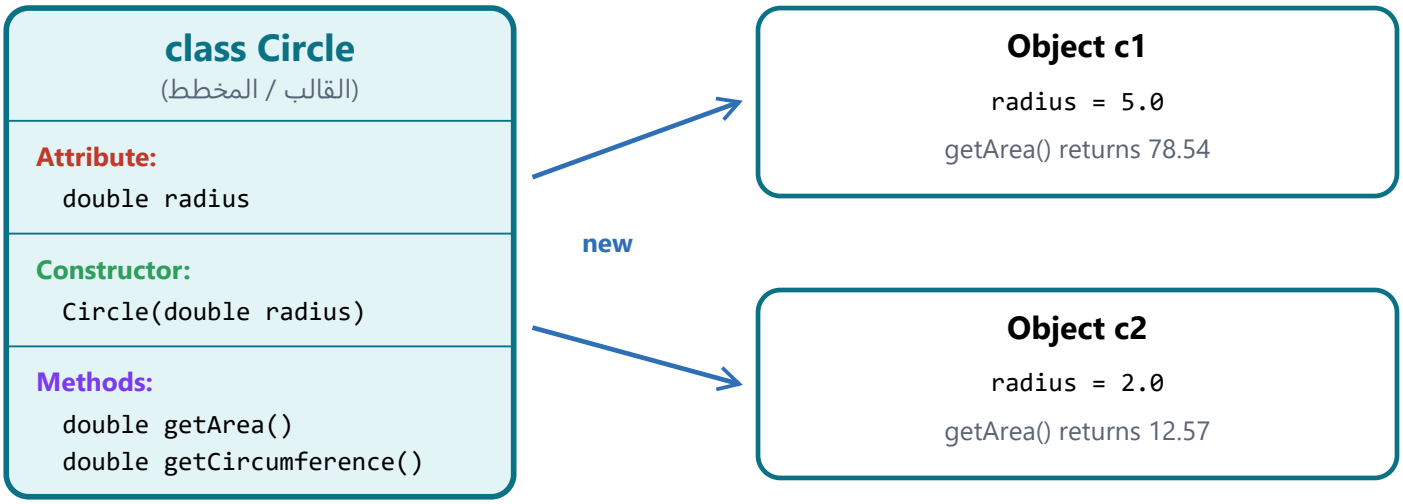
الخلاصة: اكتب جملة تصف الواقع، ثم صنّف كل كلمة مهمة: شيء كامل ← صف · معلومة ← خاصية · عمل ← تابع .

❓ **فكّر:** جرّب بنفسك: "السيارة لها لون وسرعة، وتستطيع أن تتحرك وأن تتوقف". صنّف الكلمات: ما الصف؟ ما الخصائص؟ ما التوابع؟
الجواب: الصف `Car` . الخصائص `color` و `speed` . التوابع `move()` و `stop()` .

3 الصف والغرض والخاصية والتابع والباني

خمس كلمات تحكم كل ما سنكتبه. سنعرّف كل واحدة بتشبيه من الحياة، ثم نراها في الكود.

الكلمة	معناها ببساطة	تشبيه من الحياة	مثالنا
صف **Class**	مخطط/قالب يصف "نوعاً" من الأشياء. لا يحمل بيانات حقيقية.	مخطط البيت عند المهندس، أو قالب الكعك، أو استمارة فارغة	`class Circle`
غرض **Object**	شيء حقيقي مبني من القالب، له قيمه الخاصة. نبنيه بـ `new` .	البيت المبني فعلاً، قطعة الكعك، الاستمارة المعبّأة	`new Circle(5)`
خاصية **Attribute**	"ما يملكه" الغرض: بياناته. لكل غرض نسخته الخاصة.	لون البيت وعدد غرفه	`radius`
تابع **Method**	"ما يفعله" الغرض أو "ما نسأله عنه". كود له اسم، ننفّذه عند النداء.	البيت: "افتح الباب"، "ما مساحتك؟"	`getArea()`
باني **Constructor**	تابع خاص ينفَّذ لحظة ولادة الغرض (`new`) ويعطيه قيمه الأولى. اسمه اسم الصف وبلا نوع إرجاع.	تسجيل المولود في النفوس: يُعطى اسمه فوراً	`Circle(double radius)`



💡 **فكرة:** الصف وحده "رسمة على الورق". لا يوجد دائرة حقيقية قبل أن أكتب `new` . وأستطيع أن أبني من نفس القالب **ألف** دائرة، كل واحدة بنصف قطر مختلف، وكلها تستعمل **نفس** الكود.

أصغر مثال: صف واحد وغرضان

Circle (مثال أولي، بدون وراثة)

```
public class Circle {
    double radius;                  // خاصية: نصف القطر
```

```
    Circle(double radius) {             // باني: يعطي القيمة الأولى
        this.radius = radius;
    }

    double getArea() {                  // تابع: يحسب ويرجع المساحة
        return Math.PI * radius * radius;
    }
}

// في main:
Circle c1 = new Circle(5);              // غرض أول
Circle c2 = new Circle(2);              // غرض ثانٍ
System.out.println(c1.getArea());       // 78.53981633974483
System.out.println(c2.getArea());       // 12.566370614359172
```

ماذا يحدث سطراً سطراً؟

السطر	ماذا يحدث؟
`new Circle(5)`	Java تحجز مكاناً في الذاكرة لدائرة جديدة، وتستدعي **الباني** مع القيمة 5، فيضع `radius = 5` خاصية هذا الغرض، و `this.radius` = الوسيط الداخل).
`Circle c1 = ...`	المتغيّر `c1` يحمل "عنواناً" يشير إلى الغرض (مثل رقم هاتف يوصلك للشخص، وليس الشخص نفسه).
`c1.getArea()`	"يا غرض c1، احسب مساحتك": ينفّذ كود `getArea` مستعملاً `radius` الخاص بـ `c1` فقط (=5). فيرجع 78.54.
`c2.getArea()`	نفس الكود، لكن مع `radius` الخاص بـ `c2` (=2). فيرجع 12.57. **كود واحد، بيانات مختلفة.**

> `this` تعني "هذا الغرض الذي أعمل داخله". نكتب `this.radius = radius` عندما يكون اسم الوسيط مثل اسم الخاصية: يسار = الخاصية، يمين = الوسيط.

4 التابع: متى ننشئه؟ ومتى يُنفَّذ؟

هذا السؤال له معنيان، وكلاهما مهم للطالب الضعف:

أ) متى أقرّر أن أكتب تابعاً؟ (التصميم)

اسأل نفسك	إذا كان الجواب نعم
هل هذا شيء **يفعله** الغرض أو أستطيع أن **أسأله** عنه؟ (فعل)	اجعله تابعاً
هل يمكن **حسابه** من خصائص الغرض الحالية؟ (مثل المساحة من نصف القطر)	اجعله تابعاً، **لا** خاصية
هل سأحتاج هذا العمل في **أكثر من مكان**؟	اكتبه مرة واحدة في تابع وناديه
هل يمكنني تسميته بفعل واضح؟ (`getArea` ، `printInfo`)	تابع جيد
هل يقوم بعمل **واحد** فقط؟	جيد. وإلا قسّميه إلى توابع

⚠️ خطأ شائع جداً: أن نجعل المساحة خاصية

كثير من الطلاب يكتبون المساحة هكذا، لأنهم يفكّرون: **"الدائرة لها نصف قطر ولها مساحة، إذن الاثنان خاصيتان"**.
يبدو الكلام منطقياً، لكنه **خطأ في التصميم**. لنرَ لماذا، بتجربة حقيقية:

CircleWrong.java (الطريقة الخاطئة)

```java
package shapes_explained.mistake_area_attribute;

public class CircleWrong {
    double radius;
    double area;

    public CircleWrong(double radius) {
        this.radius = radius;
        this.area = Math.PI * radius * radius;
    }

    public void setRadius(double radius) {
        this.radius = radius;
    }
}
```

خطأ: معلومة مكرّرة، يمكن حسابها من `radius`

حُسبت مرة واحدة فقط عند البناء

غيّرنا `radius` ونسينا تحديث `area` !

CircleRight.java (الطريقة الصحيحة)

```java
package shapes_explained.mistake_area_attribute;

public class CircleRight {
    private double radius;

    public CircleRight(double radius) {
        this.radius = radius;
    }

    public void setRadius(double radius) {
        this.radius = radius;
    }

    public double getArea() {
        return Math.PI * radius * radius;
    }
}
```

نخزّن فقط ما لا نستطيع حسابه

لا شيء آخر نحدّثه: لا توجد نسخة ثانية

تُحسب من `radius` الحالي دائماً

Main.java: نقارن بينهما

```java
package shapes_explained.mistake_area_attribute;

/**
 * مقارنة بين: المساحة كخاصية (خطأ) والمساحة كتابع (صحيح).
 */
public class Main {
    public static void main(String[] args) {
        CircleWrong wrong = new CircleWrong(5);
        CircleRight right = new CircleRight(5);
```

```
        System.out.println("--- radius = 5 ---");
        System.out.println("area as attribute : " + wrong.area);
        System.out.println("area as method    : " + right.getArea());

        // غيّرنا نصف القطر في الدائرتين إلى 10
        wrong.setRadius(10);
        right.setRadius(10);

        System.out.println("--- radius changed to 10 ---");
        System.out.println("area as attribute : " + wrong.area + "   <-- OLD value (wrong!)");
        System.out.println("area as method    : " + right.getArea() + "   <-- correct");
    }
}
```

المخرجات الفعلية

```
--- radius = 5 ---
area as attribute : 78.53981633974483
area as method    : 78.53981633974483
--- radius changed to 10 ---
area as attribute : 78.53981633974483   <-- OLD value (wrong!)
area as method    : 314.1592653589793   <-- correct
```

انظر إلى النتيجة: بعد أن غيّرنا نصف القطر إلى 10، الدائرة الخاطئة ما زالت تقول إن مساحتها **78.53...** (وهذه مساحة نصف القطر 5)! والدائرة الصحيحة قالت **314.15...** والأخطر: **لم يظهر أي خطأ**، البرنامج يعمل ويعطي جواباً غلطاً بثقة.

تتبّع ما حدث خطوة بخطوة

الخطوة	الكود	radius	area المخزَّنة (خاصية)	getArea() (تابع)
1	`new CircleWrong(5)`	5	78.54 ✓ (حُسبت الآن)	78.54 ✓
2	`setRadius(10)`	10	**78.54 ✗ (بقيت كما هي!)**	314.16 ✓ (تُحسب من 10)
3	نطبع المساحة	10	**78.54 ✗ خطأ**	314.16 ✓ صحيح

💡 **فكرة:** تشبيه: المساحة المخزَّنة كخاصية مثل **رقم كتبته على ورقة** بعد أن ضغطت "يساوي" في الآلة الحاسبة. لو غيّرت الأرقام بعدها يبقى الرقم المكتوب على الورقة قديماً. أما التابع فهو **أن تضغط "يساوي" من جديد في كل مرة**، فالجواب يطابق الأرقام الحالية دائماً.

لماذا هذا خطأ؟ أربعة أسباب

#	السبب	الشرح
1	معلومة مكرّرة (نسختان)	نصف القطر والمساحة يحملان **نفس المعلومة** بشكلين (المساحة تُشتق من نصف القطر بقانون). عندما نخزّن النسختين قد **تختلفان** عن بعضهما، ولا نعرف أيهما الصحيحة.
2	نسيان التحديث	في كل مكان يتغيّر فيه نصف القطر (الباني، `setRadius`، أي تعديل مباشر) يجب أن نتذكّر تحديث `area`. ننسى مرة واحدة فيظهر جواب غلط بلا أي رسالة خطأ (مثل مشكلة `return 0.0` تماماً).
3	يمكن إدخال قيمة كاذبة	لو جعلنا المساحة وسيطاً في الباني `new CircleWrong(5, 3.0)` فسنسمح بدائرة نصف قطرها 5 ومساحتها 3! الصحيح ألا نترك أحداً يكذب علينا.
4	لا حاجة لها أصلاً	حسابها يأخذ جزءاً من ألف من الثانية. لا نربح شيئاً من تخزينها، ونخسر الأمان.

القاعدة: كيف أفرّق بين الخاصية والتابع؟

اسأل نفسك: **هل أستطيع حساب هذه المعلومة من معلومات أخرى موجودة عندي؟**
✓ نعم ← **تابع** (معلومة "مشتقة": نحسبها عند الطلب).
✗ لا ← **خاصية** (معلومة "أساسية": يجب أن يعطيني إياها أحد).

المعلومة	هل أحسبها من غيرها؟	كيف؟	إذن
نصف القطر `radius`	لا	يجب أن يعطيني إياه المستخدم	**خاصية**
المساحة `area`	نعم	π × r × r	**تابع** `getArea()`
المحيط `circumference`	نعم	π × r × 2	**تابع** `getCircumference()`
سنة الميلاد `birthYear`	لا	معلومة أساسية	**خاصية**
العمر `age`	نعم	السنة الحالية − سنة الميلاد (وتتغير كل سنة!)	**تابع** `getAge()`
اسم الطالب الأول والأخير	لا	معلومات أساسية	**خاصيتان**
الاسم الكامل	نعم	الأول + الأخير	**تابع** `getFullName()`
علامات الطالب `marks`	لا	معلومات أساسية	**خاصية**
المعدل `average`	نعم	مجموع العلامات ÷ عددها	**تابع** `getAverage()`
سعر القطعة والكمية	لا	معلومات أساسية	**خاصيتان**
السعر الإجمالي	نعم	السعر × الكمية	**تابع** `getTotal()`

❓ **فكّر:** اختبر نفسك: في صف `Student` هل نجعل `average` (المعدل) خاصية أم تابعاً؟
الجواب: تابع `getAverage()`. لأن المعدل يُحسب من العلامات (مجموع العلامات ÷ عددها)، فهو معلومة مشتقة. الخاصية هي **العلامات** نفسها. ولو جعلنا المعدل خاصية ثم أضفنا علامة جديدة، سيبقى المعدل قديماً (نفس خطأ المساحة تماماً).

الخلاصة: الخاصية = معلومة أساسية نخزّنها. التابع = شيء نحسبه أو نفعله عند الطلب. وما يمكن حسابه لا نخزّنه، حتى لا يصبح قديماً أو متناقضاً.

ملاحظة للمتقدمين: في برامج كبيرة قد نخزّن ناتجاً مشتقاً لتوفير الوقت إذا كان حسابه ثقيلاً جداً، لكن بشرط أن يُحدَّث تلقائياً. في موادنا الآن: **لا نخزّن ما يمكن حسابه**.

ب) متى يُنفَّذ التابع؟ (وقت التشغيل)

- **الكتابة:** نكتب التابع **مرة واحدة** داخل الصف. وهو مجرد وصفة مكتوبة، لا يعمل بعد.
- **التنفيذ:** لا يعمل إلا عندما **نناديه** بـ `اسم_الغرض.اسم_التابع()`. وكل نداء ينفّذه من جديد.
- **العودة:** عند `return` تُرجَع النتيجة إلى المكان الذي ناديناه منه، ويكمل البرنامج من هناك.

💡 **فكرة:** التابع مثل وصفة طبخ في دفتر. كتابتها مرة واحدة لا تطبخ شيئاً. تطبخ فقط عندما تقول "اطبخ المحشي الآن". وتستطيع أن تطلب نفس الوصفة عشر مرات.

تشريح التابع

```
public  double  getArea  (  )  {
  ^      ^        ^     ^     ^
```

```
    |      |        |      |      +-- الجسم: ما ينفّذه التابع
    |      |        |      +-- الوسائط (ما يستقبله): هنا لا شيء
    |      |        +-- الاسم (فعل واضح)
    |      +-- نوع ما يرجعه (double = رقم كسري، void = لا يرجع شيئاً)
    +-- مستوى الوصول (public = يراه الجميع)
    return Math.PI * radius * radius;      // أرجع النتيجة لمن ناداني = return
}
```

ثلاثة أنواع من التوابع رأيناها في التمرين:

النوع	مثاله	ماذا يفعل
باني (Constructor)	`Circle(double radius)`	يُنفَّذ تلقائياً عند `new` فقط، مرة واحدة لكل غرض
تابع get	`getRadius()`	يعطي قيمة خاصية `private` للخارج (الخارج لا يصل لها مباشرة)
تابع حساب/عمل	`printInfo()` ، `getArea()`	يحسب أو يطبع

5 نكتب ثلاثة صفوف منفصلة ونرى المشكلة

الآن نُسقط الواقع على البرمجة: **كل شكل = صف**. أول فكرة تخطر لنا: نكتب ثلاثة صفوف، كل صف فيه ما يخصه.

ثلاثة صفوف منفصلة (بلا أب)

```
class Circle {
    double radius;
    double getArea()          { return Math.PI * radius * radius; }
    double getCircumference() { return 2 * Math.PI * radius; }
    void   printInfo()        { System.out.println("Circle area = " + getArea()); }
}
class Rectangle {
    double x, y;
    double getArea()          { return x * y; }
    double getCircumference() { return 2 * (x + y); }
    void   printInfo()        { System.out.println("Rectangle area = " + getArea()); }
}
class Square {
    double side;
    double getArea()          { return side * side; }
    double getCircumference() { return 4 * side; }
    void   printInfo()        { System.out.println("Square area = " + getArea()); }
}
```

تأملوا: **أسماء التوابع الثلاثة نفسها في كل صف** (`getArea` و `getCircumference` و `printInfo`)، وهذا منطقي لأن الواقع قال لنا: كل شكل له مساحة ومحيط. لكن الصفوف لا تعلم بوجود بعضها، فتكون المشكلة في `main` :

main بدون أب مشترك

```
// بدون أب مشترك: main لا تستطيع التعامل مع "أي شكل"
Circle    c = null;     // متغير خاص بالدوائر
Rectangle r = null;     // متغير خاص بالمستطيلات
Square    s = null;     // متغير خاص بالمربعات

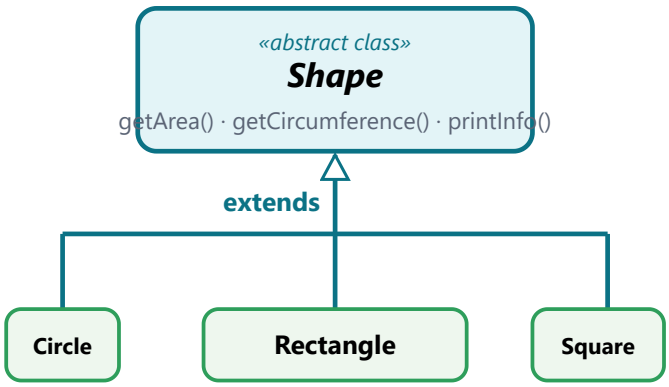
if (choice == 1)      { c = new Circle(...);    c.printInfo(); }
else if (choice == 2) { r = new Rectangle(...); r.printInfo(); }
else if (choice == 3) { s = new Square(...);    s.printInfo(); }
// وكل ما أضفنا شكلاً جديداً نضيف متغيراً و if جديدة!
```

المشكلة: لا أستطيع أن أقول "أعطني أي شكل". يجب أن أعرف نوعه مسبقاً. وكلما أضفتُ مثلثاً أو معيناً يجب أن أعدّل `main` من جديد. والبرنامج الحقيقي فيه آلاف الأسطر.

6 الفكرة الكبرى: نرفع المشترك إلى الأب Shape

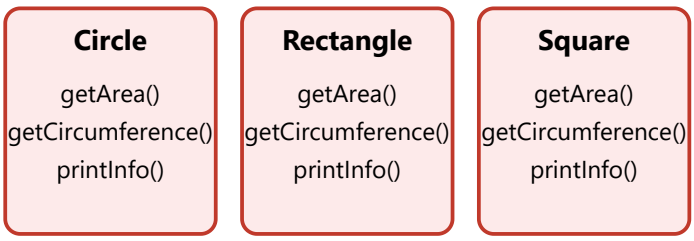
تذكّروا ما لاحظناه من الواقع: **كل شكل له مساحة ومحيط**. هذه صفة مشتركة، فلا نكرّرها في كل صف، بل **نرفعها إلى صف أب** اسمه `Shape`. وكل شكل "**هو**" نوع من Shape، فيرث منه.

Shape بعد: نرفع المشترك إلى الأب



كل ابن يكتب طريقته الخاصة بنفس الأسماء
تعمل بحلقة واحدة main و

قبل: كل صف يكرّر نفس الأفكار



ثلاثة صفوف لا يجمعها شيء
لكل نوع if تحتاج main والـ

اختبار الوراثة بكلمة "هو": الدائرة هي شكل ✓. المستطيل هو شكل ✓. إذن `Circle extends Shape` . (أما "السيارة هي دائرة"؟ لا ✗. فلا وراثة.)

ماذا نرفع؟ وماذا نترك؟

يُرفع إلى الأب Shape (مشترك)	يبقى عند كل ابن (مختلف)
وجود `getArea()` و`getCircumference()` و`printInfo()` كـ "أسماء متفق عليها"	الخصائص (`x` ، `radius` و`y`، `side`)، والباني، و**طريقة** حساب كل تابع

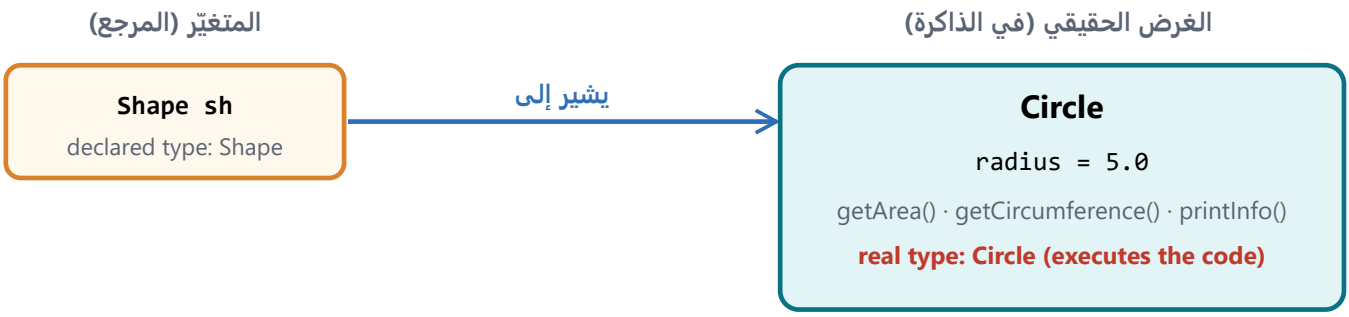
💡 **فكرة:** الأب يقول: "كل شكل **لازم** يعرف يجاوب عن مساحته ومحيطه". أما **كيف** يجاوب: هذه مسؤولية كل ابن. لأن الأب لا يعرف: هل أنا دائرة أم مربع؟

ما فائدة هذا؟ تعدد الأشكال

صار عندنا قاعدة جميلة: **متغيّر من نوع الأب يستطيع أن يحمل أي ابن.**

```
متغير Shape يحمل أبناء مختلفين
```

```
Shape sh;                  // متغير من النوع الأب
sh = new Circle(5);        // دائرة هي "شكل" ← مسموح
sh.printInfo();            // يُنفَّذ printInfo الخاص بالدائرة
sh = new Square(3);        // الآن مربع
sh.printInfo();            // يُنفَّذ printInfo الخاص بالمربع (نفس السطر، نتيجة مختلفة)
```



Java تنظر إلى **الغرض الحقيقي** (Circle) لتعرف أي `printInfo` تُنفِّذ، لا إلى نوع المتغير (Shape). لذلك السطر `sh.printInfo()` نفسه يعطي نتيجة مختلفة حسب الغرض الذي يحمله `sh` . وهذا هو **تعدد الأشكال**.

7 كتابة الصف Shape: لماذا نجعله abstract؟

الآن نكتب الصف الأب. سؤال طبيعي: **ما الذي نكتبه داخل `getArea()` في الصف `Shape`؟** الأب لا يعرف كيف يحسب المساحة، لأنه لا يعرف: هل أنا دائرة أم مربع؟ المساحة تُحسب بقانون مختلف لكل شكل.

الحل: نترك التابع **بلا جسم** ونُعلنه `abstract` . كلمة **abstract** معناها "مجرّد / غير مكتمل عمداً"، ونستعملها في مكانين:

- **تابع مجرد:** نكتب اسمه فقط وننهيه بـ `;` : "هذا مطلوب لكني لا أعرف كيف".
- **صف مجرد:** أي صف فيه تابع مجرد يجب أن يُعلَن `abstract` أيضاً.

Shape.java

```
package shapes_explained.v2_abstract;

public abstract class Shape {

    public abstract double getArea();

    public abstract double getCircumference();

    public abstract void printInfo();
}
```

الصف مجرد: لا نستطيع كتابة `new Shape()`

تابع مجرد: بلا `{ }` وينتهي بـ `;` مباشرة

كل ابن مُلزَم بكتابته، وإلا لن يُترجم الكود

كل شكل يعرف كيف يطبع معلوماته

💡 **فكرة:** الأب هنا يشبه "دفتر شروط": يقول لكل ابن: "إذا أردت أن تكون شكلاً، فعليك أن تكتب طريقتك لحساب المساحة والمحيط وطباعة المعلومات. وإلا لن أقبلك".

الابن يكتب طريقته: الدائرة

Circle.java

```java
package shapes_explained.v2_abstract;

public class Circle extends Shape {
    private double radius;

    public Circle(double radius) {
        this.radius = radius;
    }

    public double getRadius() {
        return radius;
    }

    @Override
    public double getArea() {
        return Math.PI * radius * radius;
    }

    @Override
    public double getCircumference() {
        return 2 * Math.PI * radius;
    }

    @Override
    public void printInfo() {
        System.out.println("Circle: Radius = " + radius + ...);
    }
}
```

الدائرة **هي** شكل، فترث من `Shape`

الخاصية الوحيدة التي تصف الدائرة

الباني: يعطي نصف القطر قيمته الأولى

`@Override` : نكتب طريقة الدائرة لحساب المساحة

وطريقتها لحساب المحيط

وطريقتها لطباعة المعلومات

@Override تعني: "أنا لا أضيف تابعاً جديداً، بل أستبدل نسخة الأب بنسخة خاصة بي، بنفس الاسم والوسائط". وإن أخطأنا في الاسم تنبّهنا Java.

ماذا يحمينا abstract؟ (أخطاء حقيقية من المترجم)

1) محاولة بناء شكل عام `new Shape()` :

```
shapes_explained\v2_abstract\Bad2.java:2: error: Shape is abstract; cannot be instantiated
public class Bad2 { public static void main(String[] a) { Shape s = new Shape(); } }
                                                                   ^
```

2) صف نسي أن يكتب `getArea` (مثل `Rectangle` هنا):

```
shapes_explained\v2_abstract\Rectangle.java:2: error: Rectangle is not abstract and does not override abstract
method getArea() in Shape
```

```
public class Rectangle extends Shape {
       ^
```

الخطأ يظهر **فوراً أثناء الكتابة** وليس عند الزبون. المترجم صار "مراقباً" يحمينا. وهذا ما نقصده بأن abstract **يُجبر الأبناء على الالتزام**.

قواعد abstract (للحفظ)

1. التابع المجرد بلا `{ }` ويُنهى بـ `;` .
2. الصف الذي فيه تابع مجرد يجب أن يكون `abstract` .
3. لا نستطيع `new` على صف مجرد، لكن نستطيع كتابة **متغير** من نوعه يحمل ابناً: `Shape sh = new Circle(5);` .
4. الابن (غير المجرد) **يجب** أن يكتب كل التوابع المجردة وإلا لن يُترجم.
5. الصف المجرد يمكن أن يحوي أيضاً توابع عادية وخصائص وباني.

تنبيه: خطأ شائع في الكتابة: `public void abstract double getArea();` لا يُترجَم. الصحيح: `public abstract double getArea();` (الكلمة `abstract` قبل نوع الإرجاع، ونوع إرجاع واحد فقط).

📝 ملاحظة: ماذا لو لم نكتب abstract؟

يستطيع بعض المبرمجين أن يكتبوا `Shape` كصف عادي، وأن يجعلوا توابعه ترجع `0.0` ثم يعيد الأبناء كتابتها. هذا **يعمل**، لكنه **غير احترافي** لأنه يخبّئ الأخطاء:

Shape.java (بـدون abstract)

```
package shapes_explained.v1_zero;

public class Shape {                                  صف عادي: يسمح بـ new Shape()

    public double getArea() {
        return 0.0;                                   مساحة وهمية! لا يوجد شكل مساحته صفر
    }

    public double getCircumference() {
        return 0.0;
    }

    public void printInfo() {
        System.out.println("Shape: Area = " + getArea() + " , Circumference = " + getCircumference());
    }
}
```

المخرجات الفعلية (Rectangle نسي كاتبه getArea)

```
Shape: Area = 0.0 , Circumference = 0.0
Rectangle area = 0.0   <-- wrong! it should be 24. No error was reported
Circle: Radius = 5.0 , Area = 78.53981633974483 , Circumference = 31.41592653589793
Rectangle: x = 4.0 , y = 6.0 , Area = 0.0 , Circumference = 20.0
Square: Side = 3.0 , Area = 9.0 , Circumference = 12.0
```

ثلاث مشاكل:

1. **نستطيع بناء "شكل" لا معنى له:** `new Shape()` يعمل ومساحته 0.0، مع أنه لا يوجد في الواقع شيء اسمه "شكل" فقط.
2. **النسيان يمرّ بصمت:** الصف الذي نسي `getArea` (مثل `Rectangle` هنا) لا يعطي أي خطأ. البرنامج يعمل ويطبع **جواباً خاطئاً** (0.0). وهذا أسوأ أنواع الأخطاء لأنه يختبئ.
3. **الرقم 0.0 كذبة:** لا يوجد شكل مساحته صفر.

أما مع abstract: المشاكل الثلاث تتحول إلى **أخطاء ترجمة واضحة** فنصلحها قبل التشغيل.

متى يصح الرقم الافتراضي؟ فقط إذا كان **منطقياً فعلاً** (مثل `getDiscount()` الافتراضي 0 لأن أغلب المنتجات بلا خصم). أما المساحة فأي رقم افتراضي كذبة، لذلك نستعمل `abstract` .

8 الحل الكامل بالكود

الملفات الخمسة في الحزمة `shapes_explained.v2_abstract` . أغلب التعليقات بالعربي لتساعد الطالب.

Shape.java

```
package shapes_explained.v2_abstract;

/**
 * النسخة الثانية (الاحترافية): الصف Shape مجرد (abstract).
 * نقول فيه: "كل شكل له مساحة ومحيط وطباعة معلومات، لكني لا أعرف كيف تُحسب،
 * وكل شكل ابن مُلزَم بأن يكتب طريقته بنفسه".
 *
 * قواعد abstract:
 * 1) الصف الذي فيه تابع مجرد يجب أن يكون abstract أيضاً.
 * 2) التابع المجرد بلا جسم: ينتهي بفاصلة منقوطة ; مباشرة.
 * 3) لا نستطيع كتابة new Shape() (لا معنى لـ "شكل" فقط).
 * 4) كل ابن (غير مجرد) مُلزَم بكتابة كل التوابع المجردة، وإلا لن يُترجم الكود.
 */
public abstract class Shape {

    public abstract double getArea();              // لاحظ: لا { } ولا return

    public abstract double getCircumference();

    public abstract void printInfo();
}
```

Circle.java

```
package shapes_explained.v2_abstract;

/**
 * الدائرة: صف عادي (غير مجرد)، لذلك يجب أن يكتب كل التوابع المجردة التي في Shape.
 */
public class Circle extends Shape {
    private double radius;          // خاصية: نصف القطر

    public Circle(double radius) {  // باني
        this.radius = radius;
    }

    public double getRadius() {     // تابع get: يعطي القيمة للخارج
        return radius;
```

```
    }

    @Override
    public double getArea() {
        return Math.PI * radius * radius;
    }

    @Override
    public double getCircumference() {
        return 2 * Math.PI * radius;
    }

    @Override
    public void printInfo() {
        System.out.println("Circle: Radius = " + radius + " , Area = " + getArea() + " , Circumference = " +
getCircumference());
    }
}
```

Rectangle.java

```
package shapes_explained.v2_abstract;

/**
 * المستطيل: الطول x والعرض y.
 */
public class Rectangle extends Shape {
    private double x;
    private double y;

    public Rectangle(double x, double y) {
        this.x = x;
        this.y = y;
    }

    public double getX() {
        return x;
    }

    public double getY() {
        return y;
    }

    @Override
    public double getArea() {
        return x * y;
    }

    @Override
    public double getCircumference() {
        return 2 * (x + y);
    }

    @Override
    public void printInfo() {
        System.out.println("Rectangle: x = " + x + " , y = " + y + " , Area = " + getArea() + " , Circumference =
" + getCircumference());
    }
}
```

Square.java

```
package shapes_explained.v2_abstract;

/**
 * المربع: ضلع واحد.
 */
public class Square extends Shape {
```

```
    private double side;

    public Square(double side) {
        this.side = side;
    }

    public double getSide() {
        return side;
    }

    @Override
    public double getArea() {
        return side * side;
    }

    @Override
    public double getCircumference() {
        return 4 * side;
    }

    @Override
    public void printInfo() {
        System.out.println("Square: Side = " + side + " , Area = " + getArea() + " , Circumference = " +
getCircumference());
    }
}
```

Main.java

```
package shapes_explained.v2_abstract;

import java.util.Scanner;

/**
 * الصف الرئيسي: المستخدم يختار نوع الشكل، ثم نبني الغرض المناسب ونطبع معلوماته.
 */
public class Main {
    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);

        Shape sh;                       // مرجع من النوع الأب: يستطيع حمل أي ابن
        double x, y;

        System.out.println("Enter: 1 for Circle, 2 for Rectangle, 3 for Square");
        int choice = input.nextInt();

        switch (choice) {
            case 1:
                System.out.println("Radius = ");
                x = input.nextDouble();
                sh = new Circle(x);     // غرض دائرة داخل مرجع Shape
                sh.printInfo();         // يُنفَّذ printInfo الخاص بالدائرة
                break;
            case 2:
                System.out.println("x = ");
                x = input.nextDouble();
                System.out.println("y = ");
                y = input.nextDouble();
                sh = new Rectangle(x, y);
                sh.printInfo();
                break;
            case 3:
                System.out.println("Side = ");
                x = input.nextDouble();
                sh = new Square(x);
                sh.printInfo();
                break;
            default:
```

```
                System.out.println("You entered a wrong number");
        }

        // قوة التجريد: مصفوفة من النوع Shape فيها أشكال مختلفة، وحلقة واحدة تعاملها كلها
        System.out.println("---- all shapes together ----");
        Shape[] all = {new Circle(1), new Rectangle(2, 3), new Square(4)};
        double total = 0;
        for (Shape s : all) {
            total += s.getArea();       // نفس السطر يحسب مساحة كل شكل بطريقته
        }
        System.out.println("Total area = " + total);
    }
}
```

ملاحظة عن المثلث: لو أردنا إضافة الصف `Triangle` له قاعدة وارتفاع فقط، فلا نستطيع حساب محيطه. إن كان `getCircumference()` مجرداً في الأب فإن المثلث بقاعدة وارتفاع **لا يحقق العقد**. لذلك استعملنا `Square` بدلاً منه (كما في التمرين 5). وللمثلث نحتاج الأضلاع الثلاثة (راجع تمرين 2 في آخر الملف).

9 تتبّع التنفيذ سطراً سطراً

المستخدم كتب `1` ثم `5`. نمشي مع البرنامج خطوة خطوة:

تقريباً Main الأرقام = أسطر) ماذا يحدث؟)

```
1  Scanner input = new Scanner(System.in);    // أداة قراءة من لوحة المفاتيح
2  Shape sh;                                  // متغير فارغ من نوع Shape
3  double x, y;
4  System.out.println("Enter: ...");         // يطبع القائمة
5  int choice = input.nextInt();              // choice = 1 ← المستخدم كتب 1
6  switch (choice) { case 1: ...              // يقفز إلى case 1
7  x = input.nextDouble();                    // x = 5.0 ← المستخدم كتب 5
8  sh = new Circle(x);                        // يُبنى غرض Circle، radius = 5.0، و sh يشير إليه
9  sh.printInfo();                            // Java تبحث عن printInfo في الغرض الحقيقي (Circle)
10   → داخل Circle.printInfo():  يطبع ثم getCircumference() ثم getArea()
11 break;                                     // نخرج من switch
```

المخرجات الفعلية (مستخدم كتب 1 ثم 5)

```
Enter: 1 for Circle, 2 for Rectangle, 3 for Square
Radius =
Circle: Radius = 5.0 , Area = 78.53981633974483 , Circumference = 31.41592653589793
---- all shapes together ----
Total area = 25.141592653589793
```

المخرجات الفعلية (مستخدم كتب 2 ثم 4 ثم 6)

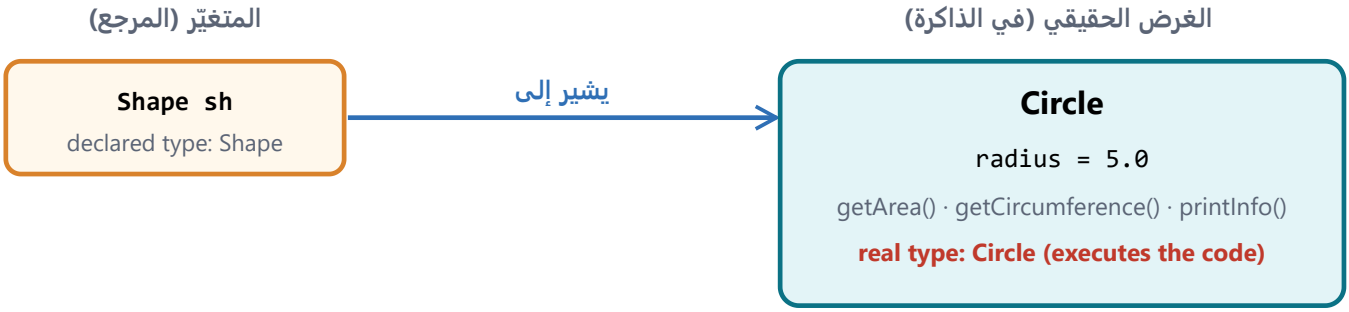
```
Enter: 1 for Circle, 2 for Rectangle, 3 for Square
x =
y =
Rectangle: x = 4.0 , y = 6.0 , Area = 24.0 , Circumference = 20.0
---- all shapes together ----
Total area = 25.141592653589793
```

المخرجات الفعلية (مستخدم كتب 7: خيار خاطئ)

```
Enter: 1 for Circle, 2 for Rectangle, 3 for Square
You entered a wrong number
---- all shapes together ----
Total area = 25.141592653589793
```

شرح السطر الذي يلخّص الدرس

`total += s.getArea();` داخل الحلقة. هذا السطر **واحد**، لكنه يحسب مساحة دائرة في الدورة الأولى ومستطيل في الثانية ومربع في الثالثة، لأن Java تسأل كل غرض: "ما مساحتك؟". النتيجة 3.14159 + 6 + 16 = 25.14159. ولو أضفنا بعد شهر `Triangle` فلن نغيّر هذا السطر أبداً.



أين كل مصطلح في هذا التمرين؟

المصطلح	أين نراه في الحل؟
صف (Class)	`Shape` و `Circle` و `Rectangle` و `Square` و `Main`
صف مجرد	`Shape`
غرض (Object)	`new Circle(x)` و `new Rectangle(x, y)` و `new Square(x)`
خاصية (Attribute)	`radius` في Circle، `x` و `y` في Rectangle، `side` في Square
باني (Constructor)	`Circle(double radius)` وأمثاله
تابع (Method)	`getArea()` و `getCircumference()` و `printInfo()` و `getRadius()`
تابع مجرد	`Shape` الثلاثة في
وراثة (extends)	`Circle extends Shape`
إعادة تعريف (@Override)	كل تابع في الأبناء يكتب طريقته
تعدد الأشكال	`Shape sh = new Circle(x); sh.printInfo();` وحلقة `for (Shape s : all)`
تغليف (private + get)	الخصائص `private` والوصول بـ `getRadius()`

10 أخطاء شائعة

الخطأ	ماذا يظهر؟	الإصلاح
كتابة `new Shape()` والصف مجرد	Shape is abstract; cannot be instantiated	ابن `new Circle(...)` وضعه في متغير `Shape`
نسيان تابع مجرد في الابن	is not abstract and does not override ... ()abstract method getArea	اكتب التابع مع `@Override`
كتابة `{ }` أو `return` داخل التابع المجرد	abstract methods cannot have a body	احذف الجسم وضع `;`
كتابة `public void abstract double ...`	خطأ في الترجمة	`public abstract double getArea();`
ظن أن المتغير يرى كل شيء في الغرض	cannot find symbol (انظر تحت)	المتغير من نوع Shape يرى فقط ما عُرّف في Shape
نسيان `this.` عند تطابق الأسماء	الخاصية تبقى 0 ولا خطأ!	`this.radius = radius;`
كتابة `1/2 * b * h` بدل `0.5 * b * h`	الناتج 0 (قسمة صحيحة)	استعمل `0.5` أو `1.0/2`

المتغير يرى نوعه فقط: لو كتبنا `Shape s = new Circle(5); s.getRadius();` :

```
shapes_explained\v2_abstract\Bad3.java:2: error: cannot find symbol
public class Bad3 { public static void main(String[] a) { Shape s = new Circle(5);
System.out.println(s.getRadius()); } }
                                                                  ^
  symbol:   method getRadius()
  location: variable s of type Shape
```

💡 **فكرة:** المتغير مثل ريموت التلفزيون: الريموت الخاص بـ Shape فيه أزرار (المساحة، المحيط، الطباعة) فقط. حتى لو كان التلفزيون الحقيقي (Circle) فيه مزايا أخرى، هذا الريموت لا يملك زراً لها.

11 تمارين للتدريب مع أجوبتها

❓ **فكّر: سؤال 1:** حلّل الجملة: "الطالب له اسم وعلامات، ويستطيع أن يحسب معدله وأن يسجّل مادة وأن يطبع بطاقته". ما الصف؟ الخصائص؟ التوابع؟

الجواب: الصف `Student` . الخصائص: `name` والعلامات `marks` . التوابع: `getAverage()` (المعدل يُحسب من العلامات فهو تابع وليس خاصية) و `registerCourse()` و `printCard()` .

❓ **فكّر: سؤال 2:** نريد إضافة `Triangle` إلى النسخة الاحترافية بقاعدة وارتفاع فقط. ما المشكلة؟ وما الحل؟

الجواب: لا نستطيع حساب المحيط من قاعدة وارتفاع (نحتاج الأضلاع الثلاثة). الحل: نجعل المثلث بثلاثة أضلاع `a, b, c` ومحيطه `a+b+c` ومساحته بقانون هيرون: `s = (a+b+c)/2` ثم `Math.sqrt(s*(s-a)*(s-b)*(s-c))` . هذه مثال أن **التجريد يكشف نقصاً في تصميمنا** مبكراً.

❓ **فكّر: سؤال 3:** لماذا لا نكتب `Shape s = new Shape();` ؟

الجواب: لأن Shape مجرد: فكرة عامة بلا طريقة حساب. لا يوجد في الواقع "شكل" بلا نوع.

❓ **فكّر: سؤال 4:** ما الفرق بين الصف والغرض؟ أعطِ مثالاً من خارج الأشكال.

الجواب: الصف قالب (مخطط)، والغرض شيء حقيقي مبني منه. مثال: `Car` قالب، و `new Car()` سيارتي الحمراء.

❓ **فكّر: سؤال 5:** لماذا نجعل الخصائص `private` ونكتب `getRadius()` ؟
الجواب: حتى لا يغيّر أحد القيمة بشكل غير منطقي (مثل نصف قطر سالب). نتحكم بها عبر توابع تتحقق من القيمة (تغليف).

❓ **فكّر: سؤال 6:** ماذا تطبع هذه الأسطر؟ `Shape a = new Square(2); System.out.println(a.getArea());`
الجواب: تطبع `4.0` ، لأن الغرض الحقيقي مربع ضلعه 2 فيُنفَّذ `getArea` الخاص بـ `Square` .

❓ **فكّر: سؤال 7:** صح أم خطأ؟ (أ) الصف المجرد يمكن أن يحوي توابع عادية. (ب) الابن يستطيع أن يتجاهل التوابع المجردة. (ج) الابن الذي هو أيضاً `abstract` لا يلزمه كتابتها.
الجواب: (أ) صح. (ب) خطأ، يجب أن يكتبها. (ج) صح: إن كان الابن نفسه مجرداً فيؤجّل الإلزام لأبنائه.

12 ملخص الدرس

1) في الواقع: كل شكل له **مساحة ومحيط**، لكن طريقة الحساب تختلف.
2) نحلّل الواقع: **الاسم ← صف أو خاصية، الفعل ← تابع، الشيء المحدد ← غرض.**
3) **الصف** قالب، **والغرض** شيء حقيقي مبني منه بـ `new` ، **والباني** يعطيه قيمه الأولى.
4) نكتب التابع **مرة واحدة**، وينفَّذ **كل مرة نناديه**. وما يمكن حسابه نكتبه تابعاً لا خاصية.
5) المشترك بين الأشكال **نرفعه إلى الأب** `Shape` ، والمختلف يبقى عند كل ابن.
6) `return 0.0` يعمل لكنه يخبّئ الأخطاء. **abstract** يُجبر كل ابن على كتابة طريقته ويمنع `new Shape()` .
7) متغير من نوع الأب يحمل أي ابن، ويُنفَّذ تابع **الغرض الحقيقي**: هذا هو **تعدد الأشكال**.

مصطلحات سريعة

الكلمة	المعنى
Class	صف: قالب يصف نوعاً من الأشياء
Object	غرض: شيء حقيقي مبني من الصف
Attribute	خاصية: بيانات الغرض
Method	تابع: ما يفعله الغرض
Constructor	باني: يُنفَّذ عند `new` ويعطي القيم الأولى
Inheritance / extends	وراثة: الابن يأخذ ما عند الأب
Override	إعادة تعريف: الابن يستبدل نسخة الأب بنسخته
Polymorphism	تعدد الأشكال: متغير الأب يحمل أي ابن، ويتصرف كلٌّ بطريقته
Abstract	مجرد: ناقص عمداً ويُلزم الأبناء بإكماله
Encapsulation	تغليف: الخصائص private والتحكم بها عبر توابع